

Document Database

Document Database: Introduction and Its Types

Introduction

A **Document Database** is a type of **NoSQL database** designed to store, retrieve, and manage **semi-structured data** as documents. These documents are typically encoded in **JSON, BSON, or XML** formats.

Each document contains pairs of **fields (keys)** and **values**, which can be of various data types, including arrays and nested documents. Document databases are schema-less or schema-flexible, making them ideal for applications with evolving or complex data models.

Key Features

- **Schema Flexibility:** No fixed schema — different documents in the same collection can have different fields.
- **Rich Querying:** Supports querying nested fields, full-text search, geospatial queries, and more.
- **High Scalability:** Built for horizontal scaling and distributed architectures.
- **Data as JSON:** Easy to map to object-oriented programming paradigms.

Advantages

- Rapid development and iteration.
- Natural fit for hierarchical or nested data.
- Supports aggregation, indexing, and advanced query operations.

Types of Document Databases

Type	Description
JSON-based	Stores documents in JSON format. Examples: CouchDB, RethinkDB.
BSON-based	Binary-encoded JSON-like storage with additional data types. Examples: MongoDB.
XML-based	Stores and queries documents using XML structure. Examples: BaseX, eXist-db.
Hybrid Document Stores	Combines document and other NoSQL models (e.g., key-value). Example: ArangoDB (multi-model).
Cloud-native Document DBs	Fully managed services offered in cloud platforms. Examples: Amazon DocumentDB, Azure Cosmos DB (Document API), Google Firestore.

Popular Document Databases

1. **MongoDB**
 - Stores data as BSON (Binary JSON).
 - Powerful aggregation framework, rich indexing, and support for ACID transactions.
2. **CouchDB**
 - Uses JSON for documents, JavaScript for MapReduce queries.
 - Designed for high availability and replication.
3. **Amazon DocumentDB**
 - Managed MongoDB-compatible database on AWS.
4. **Firebase Firestore**
 - Real-time NoSQL document database by Google for mobile/web apps.
5. **ArangoDB**
 - Multi-model database supporting documents, graphs, and key-value storage.

CouchDB is an **open-source NoSQL database** developed by Apache that is designed for **ease of use, scalability, and high availability**. It stores data in a **document-oriented** format using **JSON**, and it allows you to interact with it using **HTTP/REST APIs**.

Key Features of CouchDB:

Feature	Description
Document-based	Data is stored in JSON documents —each with its own structure and fields.
HTTP/RESTful API	You can access and manipulate data via standard web protocols . No complex drivers needed.
MVCC (Multi-Version Concurrency Control)	Avoids data conflicts and provides safe concurrent access .
Schema-free	Documents don't need to follow a fixed schema, offering flexibility in data modeling.
Replication & Synchronization	Supports offline-first apps and automatic sync between multiple instances (great for mobile/web).
MapReduce Views	You use JavaScript to define views that query and aggregate data using MapReduce .
Fault-tolerant and distributed	Built for high availability and resilience in distributed environments.

Use Cases:

- Offline-capable mobile applications (e.g., healthcare apps in remote areas)
- Content management systems
- Real-time collaborative tools
- Decentralized systems where syncing is critical

Example JSON Document in CouchDB:

```
{
  "_id": "user123",
  "name": "Jack",
  "role": "Java Developer",
  "department": "IT"
}
```

CouchDB vs Traditional RDBMS:

Feature	CouchDB	Relational DB (e.g., PostgreSQL)
Data Model	Document (JSON)	Table (Rows & Columns)
Schema	Flexible / Schema-less	Fixed Schema
Queries	MapReduce & Views	SQL
Sync & Replication	Built-in, offline-first	Limited or manual setup
Use Case	Web/mobile, NoSQL apps	Structured business data

To create a database using PUT:

```
C:\Users>curl -X PUT http://admin:1234@127.0.0.1:5984/student21
```

Note: The above command creates student21 database

o/p:

```
{"ok":true}
```

To create a database using Fauxton:

Step 1: Log in

Log In to CouchDB

Enter your username and password

admin

....

Log In

Fauston on
Fastron
v. 3.4.2
[More Info]

Log Out

Step 2: Click on right corner create database

Databases

Database name

Create Database

{ } JSON

Log Out

Name	Size	# of Docs	Partitioned	Actions
_replicator	0 bytes	0	No	
_users	2.3 KB	1	No	
admin	0 bytes	0	No	
db	374 bytes	1	No	
demo	0 bytes	0	No	
mdb	351 bytes	1	No	
mj	3.6 KB	8	No	
music	5.2 KB	5	No	
mydata	2.2 KB	5	No	
mydb12	355 bytes	1	No	
pycouchdb	434 bytes	0	No	
std	0 bytes	0	No	
std12	0 bytes	0	No	
stddb	2.5 KB	5	No	
student	374 bytes	1	No	
student21	2.0 KB	3	No	

Showing 1–20 of 22 databases. Databases per page 20

1 2

Step 3: write database name and click on bottom right create button

The screenshot shows the MongoDB Compass interface. The 'Databases' tab is active, displaying a table of databases. A 'Create Database' modal is open on the right, showing the 'Database name' field with 'bmsce' entered and the 'Partitioning' section with 'Non-partitioned' selected.

Name	Size	# of Docs	Partitioned	Action
_replicator	0 bytes	0	No	
_users	2.3 KB	1	No	
admin	0 bytes	0	No	
db	374 bytes	1	No	
demo	0 bytes	0	No	
mdb	351 bytes	1	No	
mj	3.6 KB	8	No	
music	5.2 KB	5	No	
mydata	2.2 KB	5	No	
mydb12	355 bytes	1	No	
pycouchdb	434 bytes	0	No	
std	0 bytes	0	No	
std12	0 bytes	0	No	
stddb	2.5 KB	5	No	
student	374 bytes	1	No	
student21	2.0 KB	3	No	

To create a document in database:

```
C:\Users>curl -X POST http://admin:1234@127.0.0.1:5984/student21 -H "Content-Type: application/json" -d '{"SRN": "S001", "Sname": "Rahul", "Degree": "BCA", "Sem": 5, "CGPA": 7.2}'
```

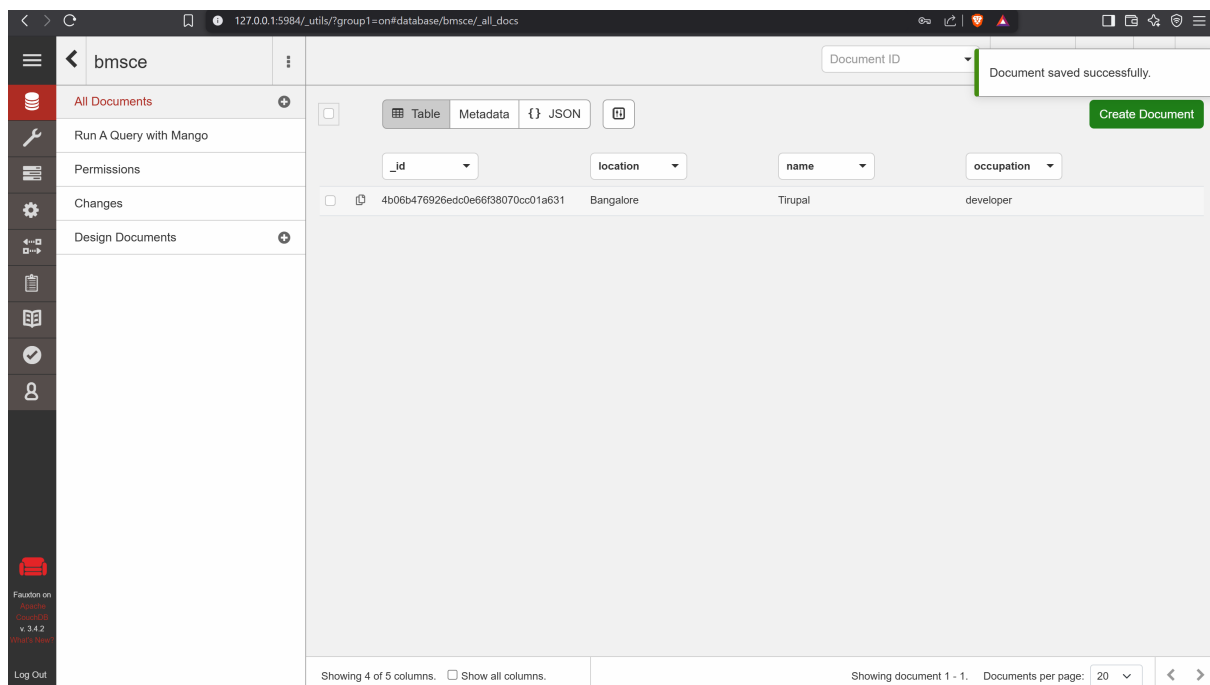
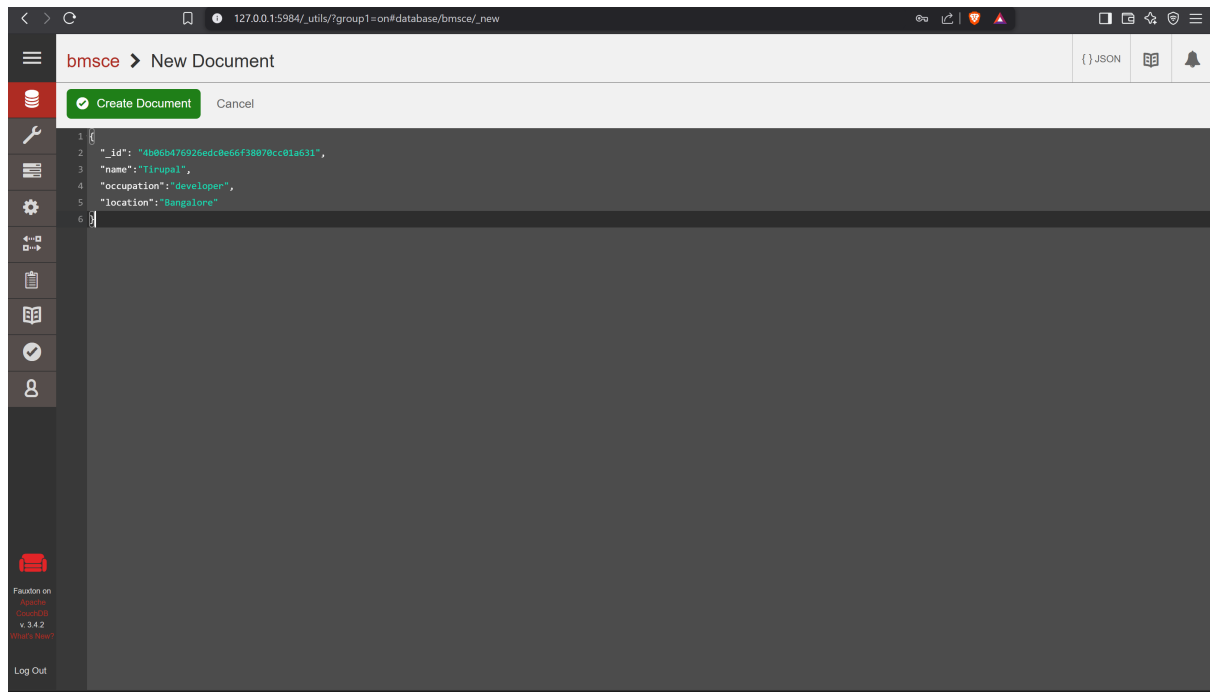
Note: The above command creates a document in student21 database

Output:

```
{"ok":true,"id":"4b06b476926edc0e66f38070cc0029da","rev":"1-ea588ab8fbe1005c8340314392ef6552"}
```

To create document in database using Fauxton :

The screenshot shows the Fauxton web interface for the 'bmsce' database. The 'All Documents' view is selected, and the main area displays 'No Documents Found' with a car icon. The left sidebar shows navigation options like 'Run A Query with Mango', 'Permissions', 'Changes', and 'Design Documents'.



To see document in the database:

C:\Users>curl http://admin:1234@127.0.0.1:5984/student21/_all_docs

Note:the above command

o/p:

```
{"total_rows":1,"offset":0,"rows":[
```

```
  {"id":"4b06b476926edc0e66f38070cc0029da","key":"4b06b476926edc0e66f38070cc0029da","value":{"rev":"1-ea588ab8fbe1005c8340314392ef6552"}}
```

```
  ]}
```

To view document using GET

C:\Users>curl -X GET http://admin:1234@127.0.0.1:5984/student21/_all_docs?include_docs=true

o/p:

```
{ "total_rows":1,"offset":0,"rows":[
{ "id":"4b06b476926edc0e66f38070cc0029da","key":"4b06b476926edc0e66f38070cc0029da","value":{ "rev":"1-
ea588ab8fbe1005c8340314392ef6552"},"doc":{"_id":"4b06b476926edc0e66f38070cc0029da","_rev":"1-
ea588ab8fbe1005c8340314392ef6552","SRN":"S001","Sname":"Rahul","Degree":"BCA","Sem":5,"C GPA":7.2}}
}]}
```

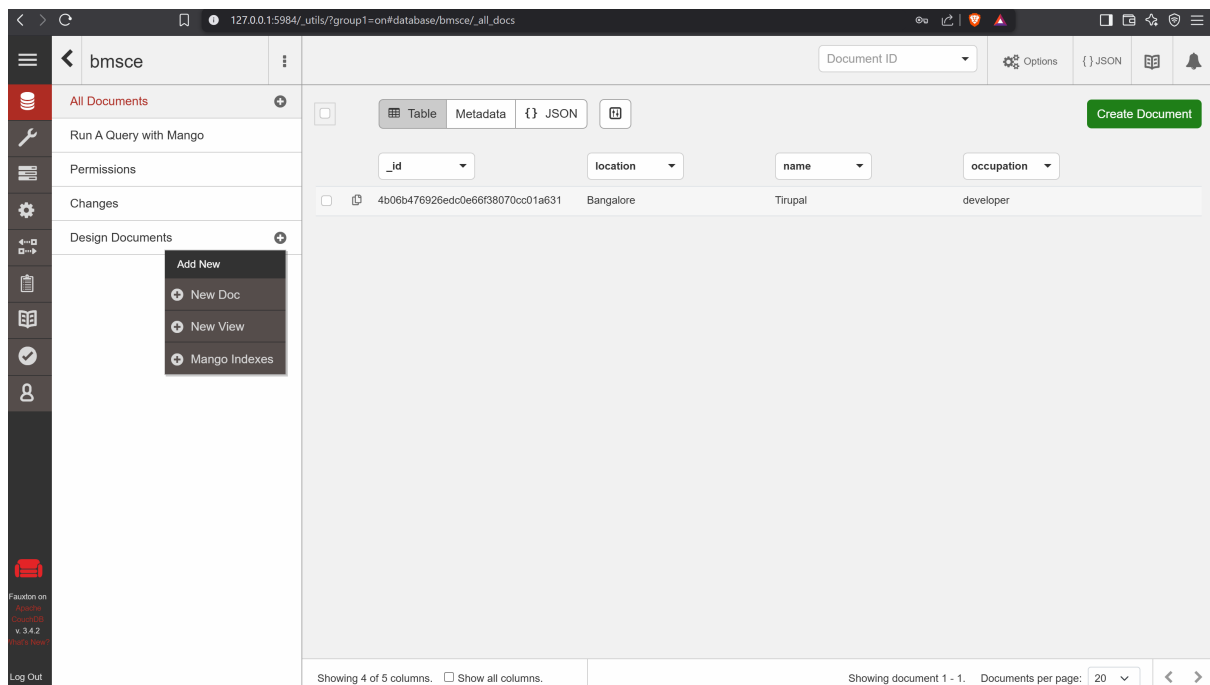
C:\Users\suresh_kannayya>curl -X PUT

http://admin:1234@127.0.0.1:5984/student21/4b06b476926edc0e66f38070cc0029da -H "Content-Type: application/json" -d "{ \"_id\": \"4b06b476926edc0e66f38070cc0029da\", \"_rev\": \"1-
ea588ab8fbe1005c8340314392ef6552\", \"SRN\": \"S001\", \"Sname\": \"Rahul\", \"Degree\":
\"BCA\", \"Sem\": 6}"

o/p:

```
{ "ok":true,"id":"4b06b476926edc0e66f38070cc0029da","rev":"2-
f28c6d340d56c883a1ac58577bb88b43"}
```

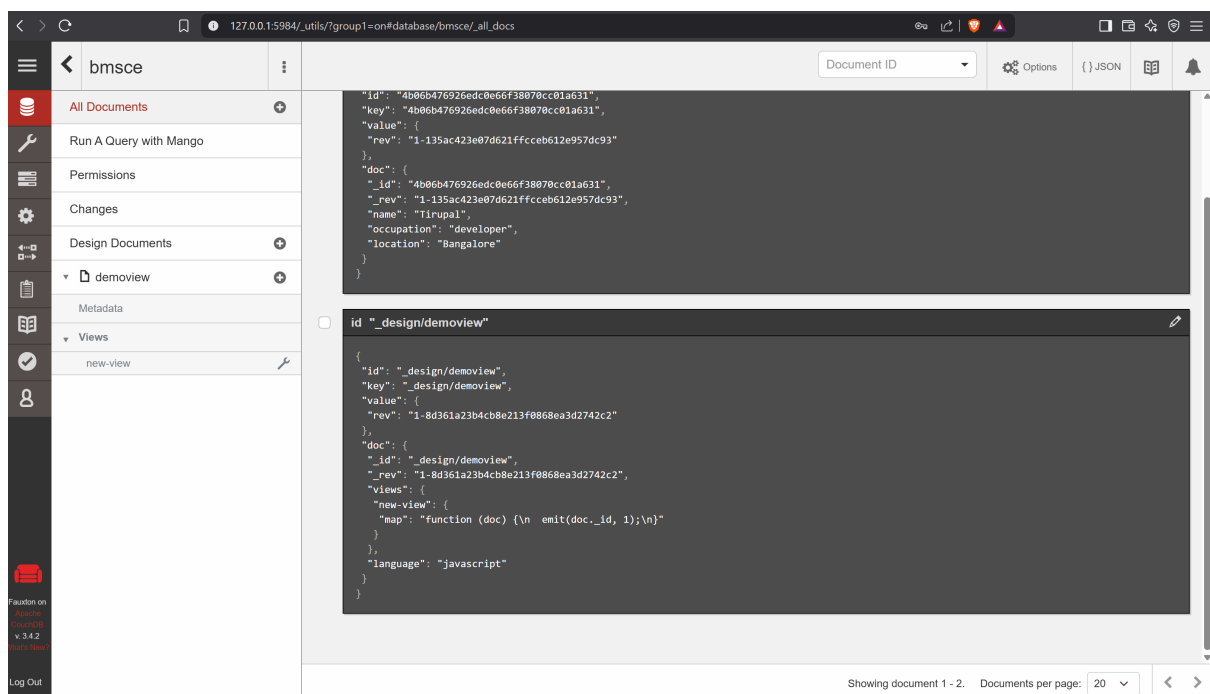
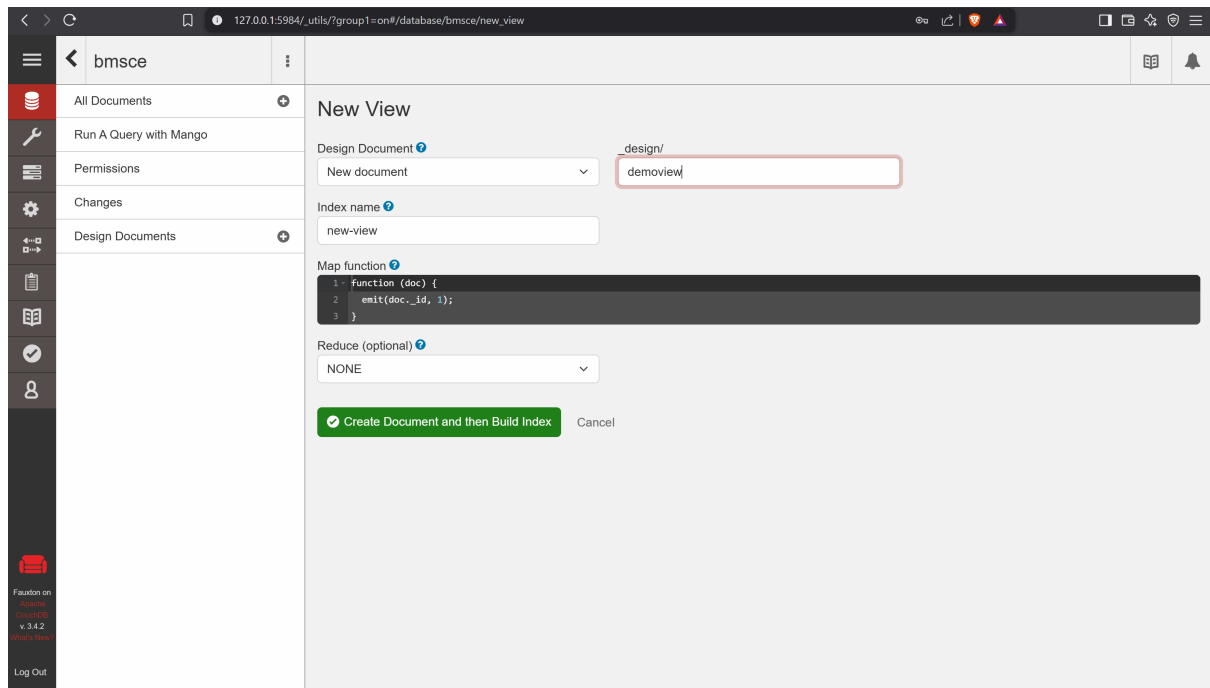
To view document using Fauxton:



The screenshot shows the Fauxton web interface for a CouchDB database named 'bmsce'. The left sidebar contains navigation options: All Documents, Run A Query with Mango, Permissions, Changes, Design Documents, and a menu for Add New (New Doc, New View, Mango Indexes). The main area displays a table view of documents. The table has columns for _id, location, name, and occupation. A single document is visible with the following details:

_id	location	name	occupation
4b06b476926edc0e66f38070cc01a631	Bangalore	Tirupal	developer

At the bottom, there are pagination controls: 'Showing 4 of 5 columns. Show all columns.' and 'Showing document 1 - 1. Documents per page: 20'.



Remove a Document with DELETE

To delete a document, use DELETE with the document's `_id` and `_rev`. This ensures the correct version of the document is deleted.

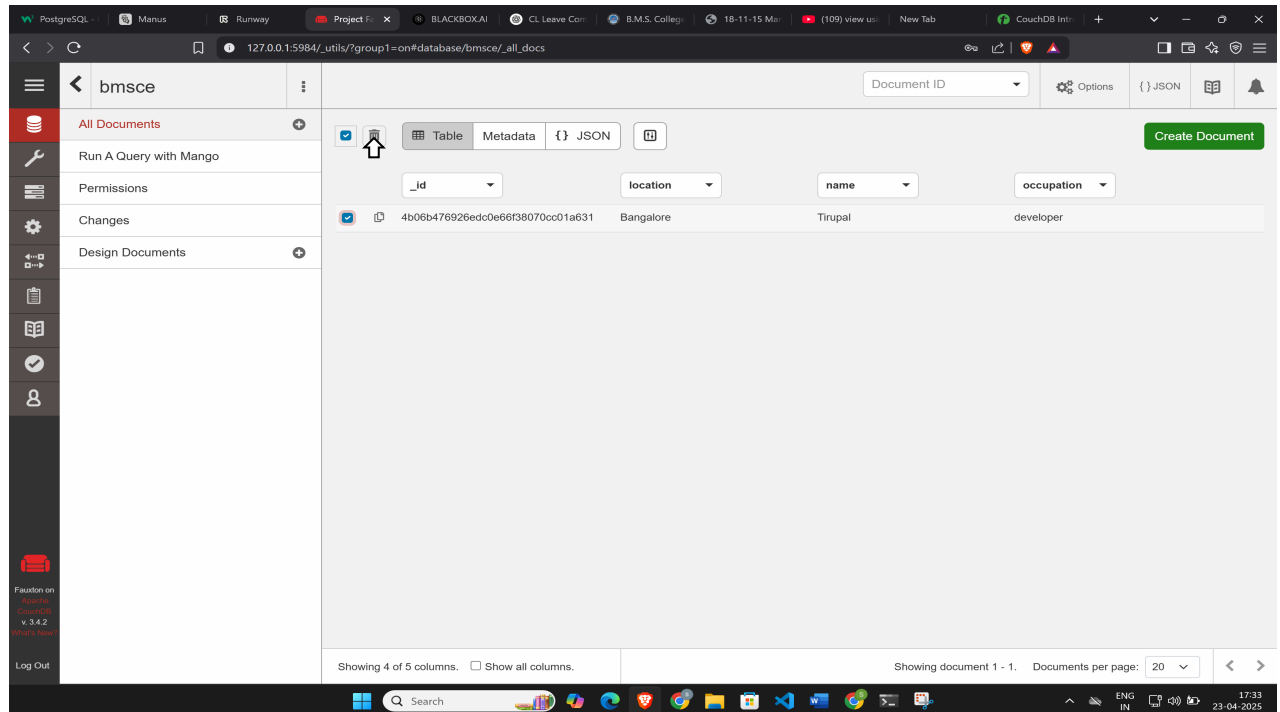
Example:

```
>curl -X DELETE
http://admin:1234@127.0.0.1:5984/student21/4b06b476926edc0e66f38070cc0029da?rev=2-f28c6d340d56c883a1ac58577bb88b43
```

Note :The above command deletes the document with `id:4b06b476926edc0e66f38070cc0029da` & `rev=2-f28c6d340d56c883a1ac58577bb88b43`

```
{"ok":true,"id":"4b06b476926edc0e66f38070cc0029da","rev":"3-8b51e29caa83d737b5386afcb729952b"}
```

Remove a Document using Fauxton:



What are Views in CouchDB?

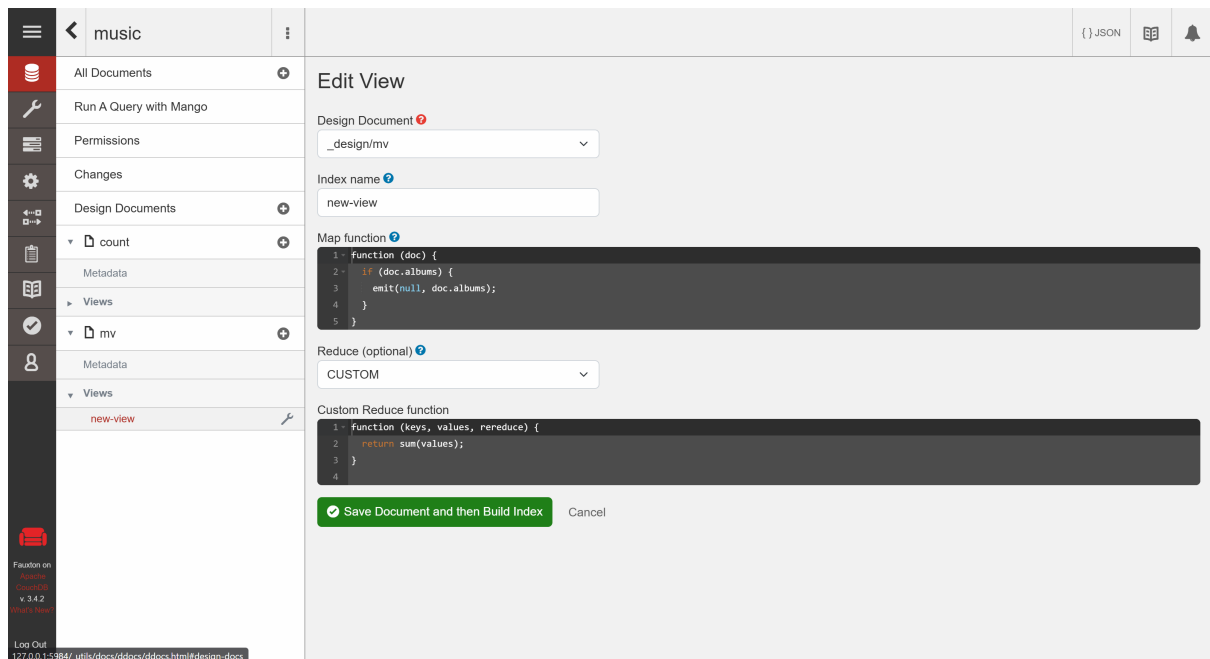
In CouchDB, a **view** is a **MapReduce-based mechanism** used to **query and index** your JSON documents.

- CouchDB does not support SQL queries.
- Instead, you use **JavaScript functions** to define how data is emitted (mapped) and optionally how it is aggregated (reduced).

Key Concepts:

Component	Purpose
Map function	Emits key-value pairs from documents (like emit(doc.name, doc.age))
Reduce function	(Optional) Summarizes or aggregates emitted values (like sum(values))
Design Document	Views are saved inside a special document (<code>_design/viewname</code>)
Indexing	Views are automatically indexed and stored for fast querying

Uses of Views in CouchDB



1. Custom Data Queries

Extract exactly the data you want from complex or nested JSON documents.

Example:

```
function (doc) {  
  emit(doc.name, doc.genre);  
}
```

Lists names and genres of all artists.

2. Aggregation and Reporting (with Reduce)

Summarize data like:

- Count of users
- Total sales per category
- Average rating per product

Example: Count Documents

```
function (doc) { emit(null, 1); }
```

Reduce:

```
function (keys, values, rereduce) { return sum(values); }
```

3. Efficient Indexing

- Views act as **indexes**, speeding up reads.

- CouchDB **caches results** and only recalculates when data changes.

Filtering and Grouping

You can use views to:

- Filter by status, category, tags, etc.
- Group data (e.g., orders per customer)

Example:

javascript

CopyEdit

```
function (doc) {  
  if (doc.status === "pending") emit(doc.customerId, 1);  
}
```

5. Real-time Sync

Views work well with the **_changes API** and **replication**, enabling real-time UI updates or syncing with mobile apps.

Where to See View Output

- In **Fauxton UI** (CouchDB's web interface)
- Using **cURL** or **HTTP GET requests**
- Embedded in **Node.js / Python apps**

In Short:

Views in CouchDB are how you **query, filter, group, and analyze** your data — they are CouchDB's **replacement for SQL SELECT queries and indexes** in relational databases.

CouchDB's Strengths and Weaknesses

CouchDB's Strengths (Advantages)

1. Document-Oriented and Schema-Free

- Stores data in **JSON documents**.
- Flexible: No fixed schema, ideal for evolving data structures.

2. Efficient MapReduce Views

- Powerful querying using **JavaScript-based MapReduce**.

- Supports complex aggregations and filtering.
- Views are **incrementally updated** (not recalculated from scratch).

3. Master-Master Replication

- Built-in support for **multi-master replication** (bi-directional).
- Ideal for **offline-first mobile apps** or multi-location setups.
- Enables syncing between devices and servers.

4. Fault-Tolerant and Durable

- Built with **append-only B+Tree storage** – minimizes risk of corruption.
- Uses **MVCC (Multi-Version Concurrency Control)** – allows concurrent reads/writes safely.

5. RESTful API

- Access everything over HTTP using simple **REST endpoints**.
- Easy to integrate with frontend and mobile apps.

6. Built-in Web UI (Fauxton)

- Manage databases, documents, and views without writing code.
- Developer-friendly for prototyping and testing.

CouchDB's Weaknesses (Limitations)

1. Slower Querying Without Indexes

- Full document scans are slow if you haven't created proper views.
- No ad-hoc querying like SQL databases (unless using Mango queries).

2. Complex Conflict Resolution

- Multi-master replication creates **document conflicts**.
- CouchDB doesn't auto-resolve – **you must handle conflict resolution manually**.

3. Eventual Consistency

- No strong ACID compliance across distributed systems.
- Favors **eventual consistency**, which may not suit financial or transactional systems.

4. Limited Aggregation Compared to SQL

- MapReduce is powerful but harder to use than SQL GROUP BY, JOIN, etc.
- No built-in relational joins or complex querying.

5. Not Ideal for Small Transactions

- For applications with **many small writes or updates**, CouchDB can become resource-heavy.
- Append-only storage can lead to disk bloat if not compacted regularly.

Use CouchDB When:

- You need **schema flexibility**
- You want **offline-first mobile syncing**
- You're building **distributed apps** with **easy replication**
- You prefer **REST APIs over SQL**

Avoid CouchDB If:

- You need **strong consistency (ACID)**
- You rely heavily on **complex joins**
- Your app needs **real-time analytics** or **fast ad-hoc querying**